

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Лабораторная работа №2

по дисциплине «Естественно-языковой интерфейс интеллектуальных
систем»

на тему «Автоматическое распознавание языка текста
(вариант 25)»

Выполнили студенты гр. 321701

Ковальчук В. Н.
Самович В. М.

Проверил

Крапивин Ю. Б.

Минск 2026

Цель: изучить и отработать практические навыки применения методов автоматического распознавания языка текстовых документов.

Задание (вариант 25): разработать систему автоматического распознавания языка текста со следующей комбинацией признаков варианта:

- **Языки текста** – французский и английский;
- **Формат документа** – HTML;
- **Реализуемые методы** – метод частотных слов, алфавитный метод и нейросетевой метод.

Согласно методическим указаниям, для каждого метода необходимо:
а) построить тренировочный набор текстов для распознаваемых языков;

б) в соответствии с методом построить поисковый образ языка (ПОЯ) на основании тренировочного набора;

в) для каждого входного документа построить поисковый образ документа (ПОД) по тем же правилам;

г) рассчитать метрику расстояния между ПОД и каждым ПОЯ и выбрать язык с минимальным значением метрики в качестве результата;

д) сравнить результаты распознавания разными методами по точности и затраченному времени.

Кроме того, система должна:

- на выходе выдавать активную ссылку на документ, результат идентификации языка для отдельного текста и сводную статистику по всей тестовой коллекции;
- предоставлять средства сохранения результата в файл и печати;
- иметь простой и доступный пользовательский интерфейс со справочными подсказками.

Структура разработанной системы

Определение языка текста реализовано как новый сценарий использования поверх уже существующей архитектуры ЛР1 (см. docs/ARCHITECTURE.md): границы сервисов по-прежнему проведены по зоне ответственности, а не по лабораторной работе, поэтому ЛР2 добавляет один новый сервис и новые таблицы БД, не создавая отдельной инфраструктуры. Структура системы на уровне контейнеров (C4, уровень 2) показана на рисунке 1; компоненты и связи, добавленные именно в этой лабораторной работе, выделены зелёным цветом и вынесены отдельным пунктом легенды.

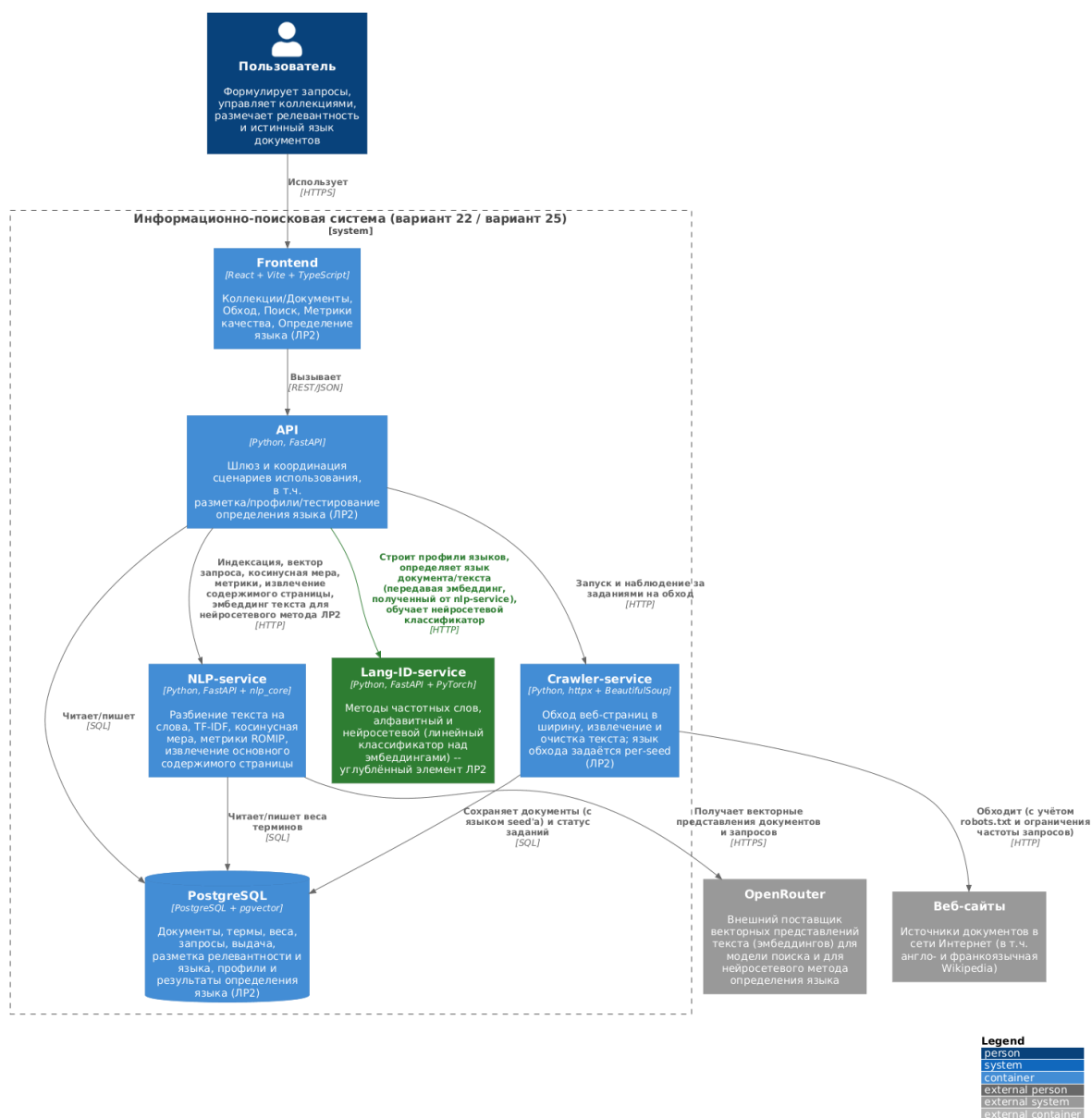


Рисунок 1 – Диаграмма контейнеров разработанной системы (нотация C4); зелёным – новое в ЛР2

Новый сервис **lang-id-service** (Python, FastAPI + PyTorch) – углублённый элемент варианта 25 (реализация методов распознавания языка). Он не хранит состояние и не обращается к БД самостоятельно: как и **nlp-service**, он получает на вход уже подготовленные данные (тексты для построения профиля, координаты весов классификатора) и возвращает чистый результат вычисления – профиль языка или решение о языке одного текста. Оркестрацией (какие профили есть, какие документы размечены, что показывать пользователю) занимается **api**, как и для остальных сценариев системы. Помимо нового сервиса, ЛР2 расширила ответственность трёх уже существующих компонентов:

– **frontend** получил новую страницу «Язык» с четырьмя вкладками

(разметка, профили, ручная проверка, тестирование);

- **crawler-service** и модель **CrawlSeed** стали поддерживать язык обхода не на уровне коллекции, а на уровне каждого стартового адреса (**CrawlSeed.language**) – это позволяет наполнять одну коллекцию сразу несколькими языковыми Wikipedia одновременно, автоматически предоставляя каждому документу язык его источника;

- в **packages/nlp_core** (переиспользуемая библиотека, общая для **nlp-service**, **crawler-service** и **api**) добавлены чистые функции построения профилей для лексических методов (**frequent_words.py**, **alphabetic.py**) и извлечение основного содержимого HTML-страницы (**content_extraction.py**) – общее и для массового краулинга, и для разовой проверки произвольного URL.

Структура базы данных

Схема ключевых таблиц и связей между ними показана на рисунке 2 (нотация Crow's Foot); таблицы, добавленные в ЛР2, выделены зелёным.

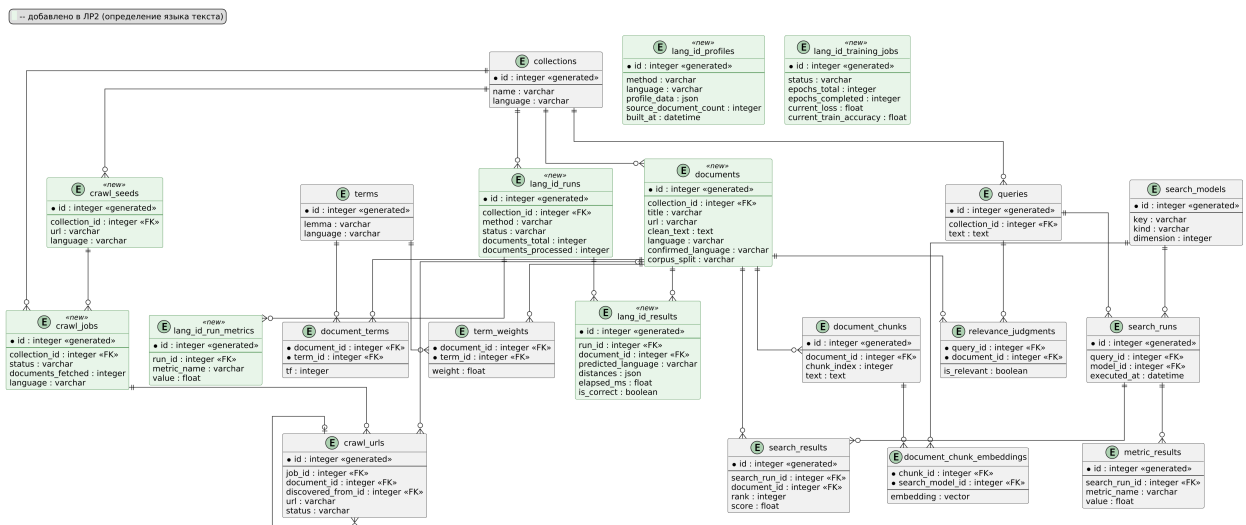


Рисунок 2 – ER-диаграмма базы данных (нотация Crow's Foot); зелёным – новое в ЛР2

Структуры данных для входной и выходной информации метода определения языка:

– **вход (обучающая коллекция)** – поле `documents.confirmed_language` хранит человеком подтверждённый истинный язык документа (это и есть эталонная разметка LP2, аналог `relevance_judgments` из LP1), а `documents.corpus_split` ($\in \{\text{train}, \text{test}, \text{NULL}\}$) относит размеченный документ к обучающей или тестовой выборке;

- **вход (обученная модель)** – таблица `lang_id_profiles`: одна строка на пару (метод, язык) для лексических методов, и одна строка с `language = NULL` для нейросетевого метода (у него один общий классификатор на все языки). Поле `profile_data` (JSON) хранит то, что нужно конкретному методу – ранжированный список слов, распределение частот букв, либо веса/смещения/порядок классов и кривую обучения нейросети; `lang_id_training_jobs` хранит текущий прогресс обучения нейросетевого классификатора (эпоха, значение функции потерь, точность на обучающей выборке) для живого прогресс-бара в интерфейсе;

– **выход (результат одного прогона тестирования)** – `lang_id_runs` (одно тестирование одного метода на тестовой выборке коллекции) и `lang_id_results` (результат по каждому документу: предсказанный язык, расстояния до профиля каждого языка, затраченное время, совпадение с `confirmed_language`);

- **выход (агрегат)** – `lang_id_run_metrics` хранит посчитанные по

прогону Accuracy/Precision/Recall/F1, по аналогии с тем, как `metric_results` кеширует метрики поиска в ЛР1.

Основные алгоритмы

Все три реализованных метода подчиняются одинаковой трёхшаговой схеме задания: построить поисковый образ языка (ПОЯ) по обучающему корпусу, построить поисковый образ документа (ПОД) по тем же правилам и выбрать язык с минимальным расстоянием между ПОЯ и ПОД. Общий алгоритм показан на рисунке 3; методы отличаются только правилом построения ПОЯ/ПОД (шаг 2) и метрикой расстояния – итоговое правило выбора (шаг 3) для всех методов одно и то же.

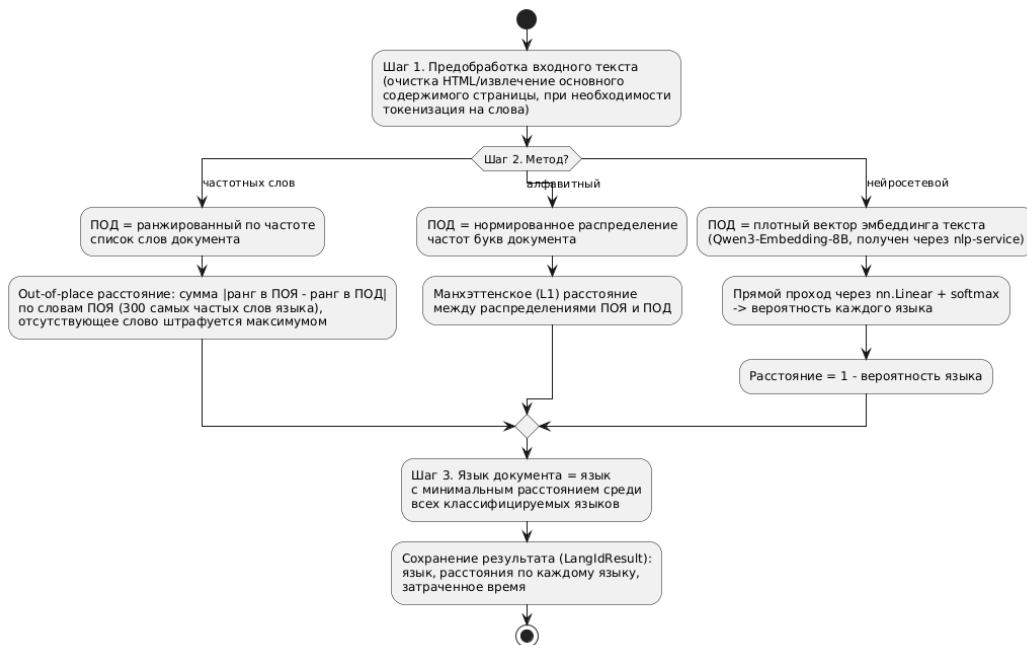


Рисунок 3 – Алгоритм определения языка одного текста (общая схема для всех трёх методов)

Метод частотных слов. ПОЯ – список из 300 самых частых слов обучающего корпуса языка (аналог 300 самых частых N-грамм из метода Кавнара–Тренкла (1994), но применённый к целым словам, а не к N-граммам символов, как того требует вариант задания). Расстояние до ПОД документа – сумма модулей разности рангов слова в ПОЯ и в частотном списке слов документа (*out-of-place measure*):

$$d_{\text{words}}(\text{ПОЯ}, \text{ПОД}) = \sum_{w \in \text{ПОЯ}} |\text{rank}_{\text{ПОЯ}}(w) - \text{rank}_{\text{ПОД}}(w)|, \quad (1)$$

где слову ПОЯ, отсутствующему в документе, назначается штраф, равный размеру самого ПОЯ (300) – он заведомо больше любого реального рассогласования рангов внутри списка.

Алфавитный метод. ПОЯ – нормированное распределение частот букв обучающего корпуса языка: $p_i = \text{count}(c_i) / \sum_j \text{count}(c_j)$ по всем алфавитным символам текста (`str.isalpha()`), без привязки к фиксированному 26-буквенному латинскому алфавиту – это существенно для французского языка, где буквы с диакритикой (é, è, ç, à, ...) несут различительную информацию и не сворачиваются в свой безакцентный вариант. Расстояние – манхэттенское (L1) между распределениями ПОЯ и ПОД:

$$d_{\text{alpha}}(\text{ПОЯ}, \text{ПОД}) = \sum_c |p_c^{\text{ПОЯ}} - p_c^{\text{ПОД}}|. \quad (2)$$

Нейросетевой метод. ПОЯ и ПОД в этом методе не строятся явно как профили языка – вместо этого один общий классификатор $\text{softmax}(W \cdot x + b)$ (линейный слой без скрытых слоёв, x – эмбединг документа Qwen3-Embedding-8B той же модели, что и дополнительная модель поиска ЛР1) обучается сразу на всех классифицируемых языках. Линейная архитектура выбрана осознанно: на нескольких сотнях документов на язык дополнительный скрытый слой не имеет статистической опоры и переобучится, тогда как линейный слой с L_2 -регуляризацией (`weight_decay` в оптимизаторе Adam) математически эквивалентен мультиномиальной логистической регрессии – разумная модель для данного объёма данных при полноценном цикле обучения (200 эпох, разбитых на чанки по 10 для живого прогресс-бара). «Расстоянием» до языка l для единообразия с двумя другими методами считается $d_{\text{neural}}(l) = 1 - P(l | x)$, так что правило «язык с минимальным расстоянием» остаётся общим для всех методов (`app.domain.lang_id`).

Оценка качества. Для каждого метода и каждого документа тестовой выборки известны истинный язык (`confirmed_language`) и предсказанный. Из пар (истинный, предсказанный) строится матрица ошибок, по которой считаются точность (Ассигасу, доля верных предсказаний), Precision/Recall/F1 отдельно для каждого языка, а затем их среднее по всем языкам (макро-усреднение) – рекомендованный подход, когда языки представлены в тестовой выборке неравными долями.

Тестовая коллекция документов

Согласно методическим указаниям, тестовая коллекция сформирована на основании многоязычных статей Wikipedia. Коллекция Wikipedia наполнена двумя независимыми запусками обхода, каждый ограничен своим доменом: с en.wikipedia.org (глубина 3, лимит 500 документов, переход только в пределах домена) и с fr.wikipedia.org (глубина 3, лимит 500 документов, переход только в пределах домена) – итого 1000 документов, каждый автоматически помечен языком своего seed'a. Каждому документу вручную подтверждён истинный язык (истинный язык у всех документов совпал с языком seed'a, из которого он получен, что ожидаемо для обхода внутри одного языкового раздела Wikipedia) и он отнесён к обучающей или тестовой выборке – кнопкой автоматического стратифицированного разбиения (80 %/20 % отдельно по каждому языку, чтобы оба языка были представлены в обеих выборках пропорционально) либо вручную по каждому документу (рисунок 4): итого 800 документов в обучающей выборке (400 на английском, 400 на французском) и 200 в тестовой (по 100 на язык).

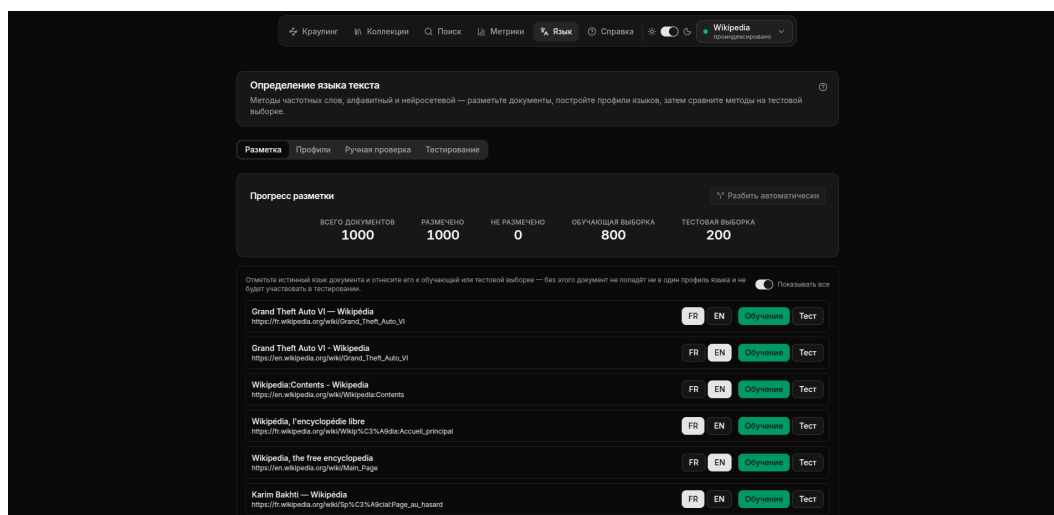


Рисунок 4 – Вкладка «Разметка»: прогресс разметки коллекции и назначение истинного языка/выборки каждому документу

По обучающей выборке построены профили обоих лексических методов для каждого языка (рисунок 5) и обучен единый нейросетевой классификатор на всех 800 обучающих документах сразу.

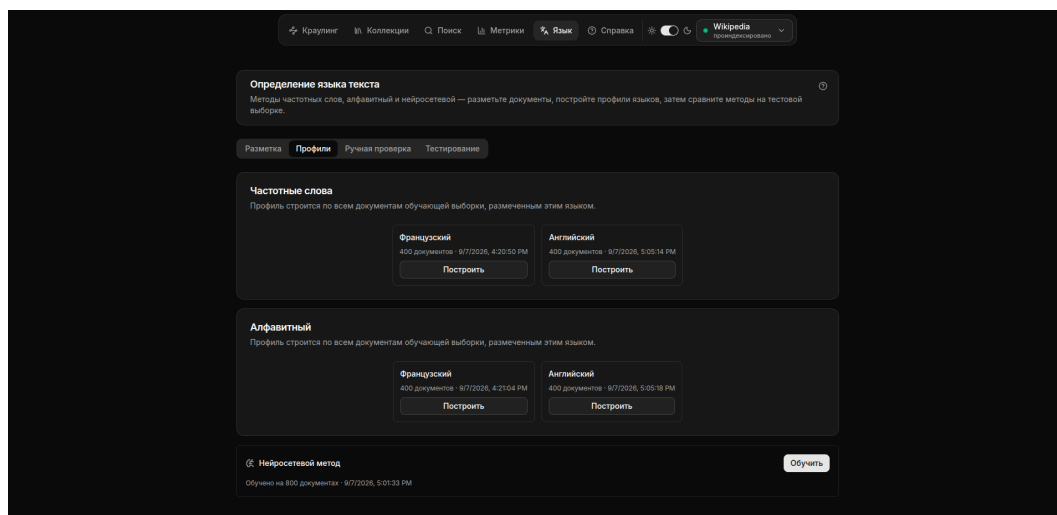


Рисунок 5 – Вкладка «Профили»: построенные профили частотных слов и алфавитные профили по каждому языку, обученный нейросетевой классификатор

Результаты тестирования и оценка качества

Каждый из трёх методов протестирован на одной и той же тестовой выборке из 200 документов (100 английских, 100 французских), не пересекающейся с обучающей. Средние по документам метрики приведены в таблице 1; тот же расчёт в интерфейсе системы – на рисунках 6 (вкладка «Тестирование») и 7 (страница «Метрики»).

Таблица 1 – Метрики качества и быстродействия по 200 документам тестовой выборки

Метрика	Частотные слова	Алфавитный	Нейросетевой
Accuracy	95,5 %	100,0 %	100,0 %
Precision (macro)	95,7 %	100,0 %	100,0 %
Recall (macro)	95,5 %	100,0 %	100,0 %
F1 (macro)	95,5 %	100,0 %	100,0 %
Среднее время классификации	3,96 мс	4,97 мс	1,24 мс

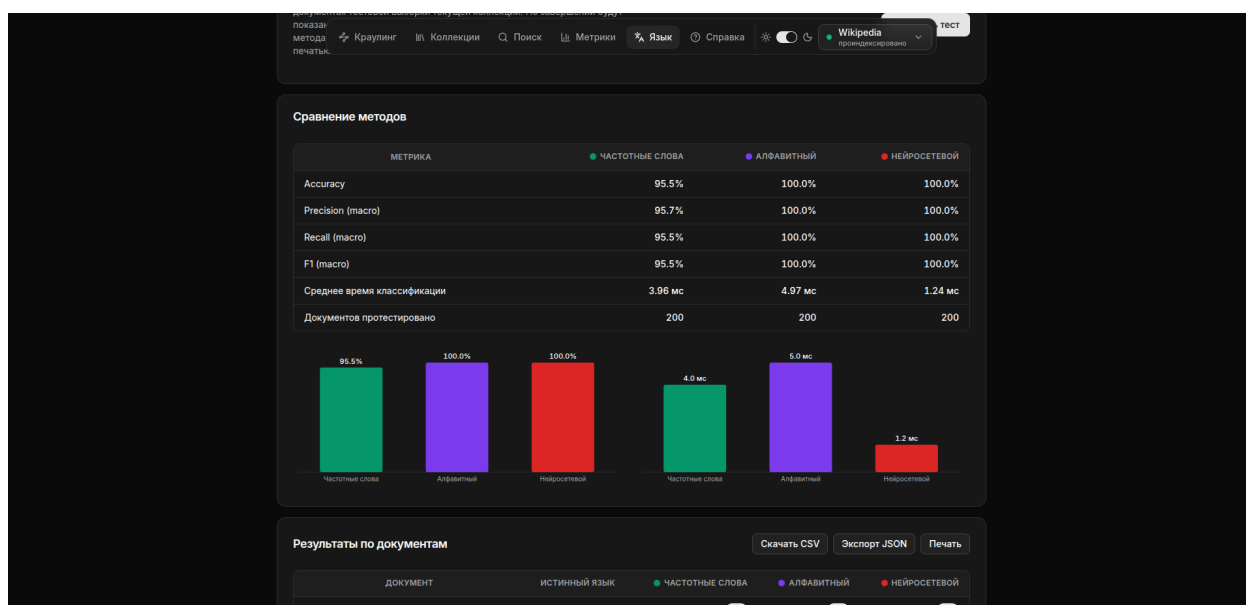


Рисунок 6 – Вкладка «Тестирование»: сравнение методов по точности и времени, результаты по документам с экспортом (CSV/JSON) и печатью

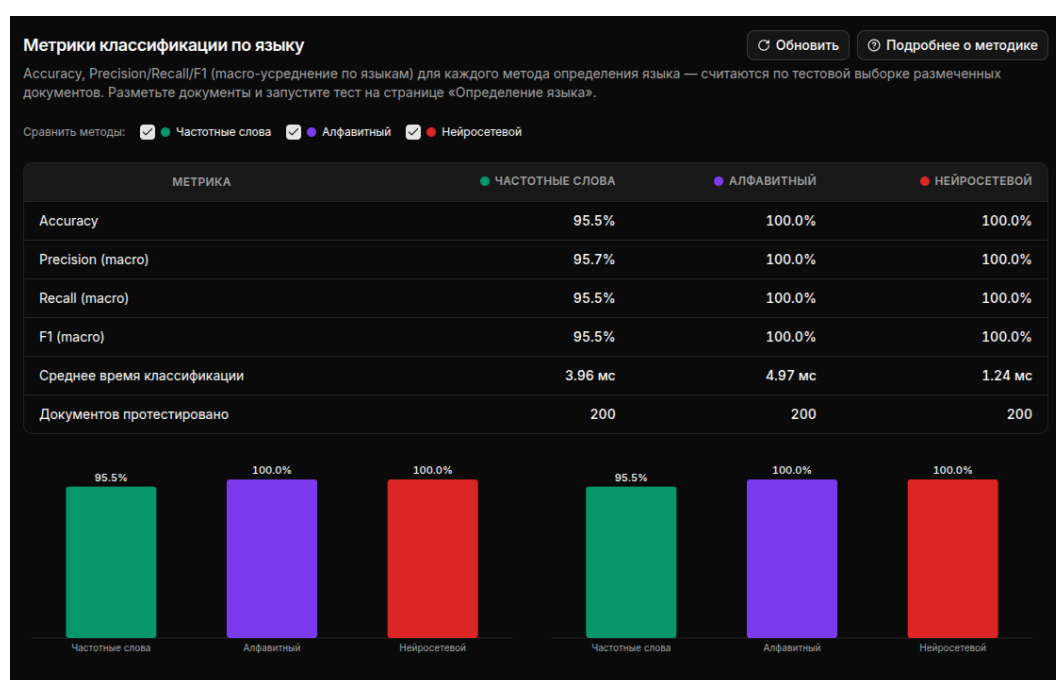


Рисунок 7 – Раздел «Метрики классификации по языку» на странице «Метрики» – тот же расчёт, что и на вкладке «Тестирование», с возможностью пересчёта

Отдельно от прогона по тестовой выборке система поддерживает разовую проверку одного текста – по адресу веб-страницы (основное содержимое которой извлекается тем же алгоритмом Readability, что и при массовом краулинге, без создания документа в коллекции) или по вставленному/загруженному тексту, сразу всеми тремя методами (рисунок 8). Результат разовой проверки, как и результат тестового прогона, можно выгрузить в CSV/JSON или распечатать.

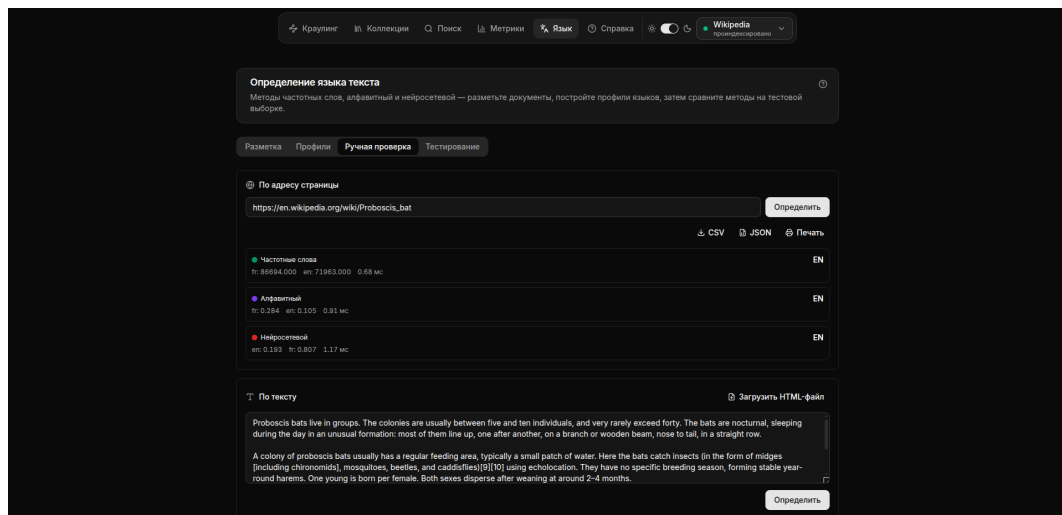


Рисунок 8 – Вкладка «Ручная проверка»: определение языка по адресу страницы всеми тремя методами с расстояниями и временем

Анализ результатов и предложения по улучшению

а) **Оба языковых профиля разошлись у метода частотных слов реже, чем у остальных двух.** На паре английский/французский алфавитный и нейросетевой методы дали 100 % точность на всех 200 тестовых документах – ожидаемый результат: у английского и французского языков заметно разное распределение частот букв (диакритические знаки, иные пропорции гласных) и разделимые в векторном пространстве эмбединги, поэтому оба сигнала оказались практически линейно разделимыми на этой паре языков. Метод частотных слов ошибся на $\approx 4,5\%$ документов – вероятная причина: фрагменты статей Wikipedia, где часто встречаются вставки на другом языке (имена собственные, цитаты, интервики-ссылки), сдвигающие ранги слов сильнее, чем распределение букв или семантику текста в целом.

б) **Нейросетевой метод при этом быстрее обоих лексических.** Среднее время классификации одного документа – 1,24 мс у нейросетевого метода против 3,96 мс (частотные слова) и 4,97 мс (алфавитный, таблица 1). Причина – у нейросетевого метода на этапе классификации выполняется единственный прямой проход через линейный слой над уже готовым эмбедингом, тогда как лексические методы заново токенизируют/анализируют посимвольно весь текст документа при каждой проверке.

в) **Что можно добавить в систему для её улучшения.**

- расширение количества классифицируемых языков (сейчас два)
- архитектура методов (профиль на язык у лексических методов, число выходных классов у нейросетевого) уже рассчитана на произвольное число языков, ограничение только в количестве размеченных данных;

– метод N-грамм символов (в дополнение к уже реализованному методу частотных слов) – по вариантам задания это тот же класс методов, что и частотные слова, но устойчивее к языкам с богатой морфологией, где словоформа реже повторяется буквально;

– доверительный порог (confidence threshold) для нейросетевого метода – отдельный класс «не удаётся определить» для документов, у которых вероятность лучшего языка низкая, вместо всегда принудительного выбора одного из известных языков;

– кросс-валидация по нескольким случайным стратифицированным разбиениям вместо одного фиксированного train/test – сейчас 100 % точность алфавитного и нейросетевого методов может быть отчасти объяснена удачным разбиением, а не только качеством методов.

г) Где систему можно применить.

– предварительная фильтрация языка документов перед их попаданием в коллекцию поиска ЛР1 – например, автоматическая маршрутизация англо- и франкоязычных страниц в отдельные индексы вместо смешанного мультязычного индекса;

– модерация многоязычных пользовательских данных (комментарии, обращения в поддержку) – быстрая маршрутизация по языку без привлечения тяжёлых моделей перевода;

– учебный полигон для последующих лабораторных работ курса, использующих понятие языка документа (машинный перевод, синтез и распознавание речи) – размеченный по языку корпус и профили уже хранятся в общей БД и переиспользуемы.

Использованные готовые компоненты

Таблица 2 – Библиотеки и программные средства, использованные при реализации ЛР2 (дополнительно к перечисленным в отчёте по ЛР1)

Компонент	Назначение
PyTorch (<code>torch.nn.Linear + softmax</code>)	Нейросетевой классификатор языка: обучение (Adam, L_2 -регуляризация) и инференс поверх готовых эмбедингов документа.
readability-lxml	Извлечение основного содержимого HTML-страницы (алгоритм Mozilla Readability, тот же, что в Firefox Reader View) перед определением языка произвольного URL и перед очисткой страниц при краулинге – убирает навигацию, подвал и рекламные блоки, оставляя только текст статьи.
Qwen3-Embedding-8B (через OpenRouter)	Источник плотных векторных представлений документа для нейросетевого метода – та же модель, что и в дополнительной модели поиска ЛР1; эмбединги, посчитанные при индексации коллекции для поиска, переиспользуются без повторного обращения к OpenRouter.

Выводы

В ходе выполнения лабораторной работы реализована система автоматического распознавания языка текста по варианту 25 (английский и французский языки, HTML-документы, методы частотных слов, алфавитный и нейросетевой) как новый сценарий использования поверх архитектуры, разработанной в ЛР1: добавлен один новый сервис (`lang-id-service`), пять новых таблиц БД и новая страница интерфейса, при этом ни один существующий контракт системы (поиск, краулинг, метрики поиска) не был нарушен. Все три метода подчиняются единой трёхшаговой схеме (построение ПОЯ по обучающему корпусу, построение ПОД для входного документа, выбор языка с минимальным расстоянием между ними), различаясь только правилом построения профиля и метрикой расстояния, что позволило реализовать общий, покрытый тестами интерфейс сравнения методов (`app.domain.lang_id`).

На собранной тестовой коллекции Wikipedia (1000 статей, поровну с `en.wikipedia.org` и `fr.wikipedia.org`, 800 обучающих / 200 тестовых

документов) алфавитный и нейросетевой методы показали 100 % точность, а метод частотных слов – 95,5 % (табл. 1) – ожидаемый результат для языковой пары со столь разными системами письма и словарным составом. По быстродействию нейросетевой метод оказался почти в три раза быстрее обоих лексических (1,24 мс против 3,96–4,97 мс на документ) за счёт того, что классификация выполняется одним прямым проходом над уже готовым эмбедингом документа, без повторного анализа всего текста. Единственная методологическая оговорка – точность 100 % у двух из трёх методов может отчасти объясняться тем, что использовалось одно фиксированное стратифицированное разбиение на обучающую и тестовую выборки, а не усреднение по нескольким случайным разбиениям (кросс-валидация); на менее контрастной паре языков или при другом разбиении разрыв между методами, вероятно, был бы заметнее. Разработанный интерфейс поддерживает как пакетное тестирование на всей размеченной коллекции, так и разовую проверку произвольной веб-страницы или текста, с выгрузкой результата в CSV/JSON и печатью, что закрывает соответствующее требование методических указаний.